UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ 3303 – INTERNET PROGRAMMING

TOPIC 2 – Java Servlet

innovative • entrepreneurial • global



UTM JOHOR BAHRU

Objectives

Applied

1. Code and test servlets that require the features presented in this chapter.
2. Use the web.xml file or an annotation (servlet 3.0 and above) to map a servlet to the URL pattern.
3. Provide for server-side data validation in your application.
4. Use the web.xml file to set initialization parameters. Use servlets to get the parameters.
5. Use the web.xml file to implement custom error handling.
6. Write debugging data for a servlet to either the console or a log file.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

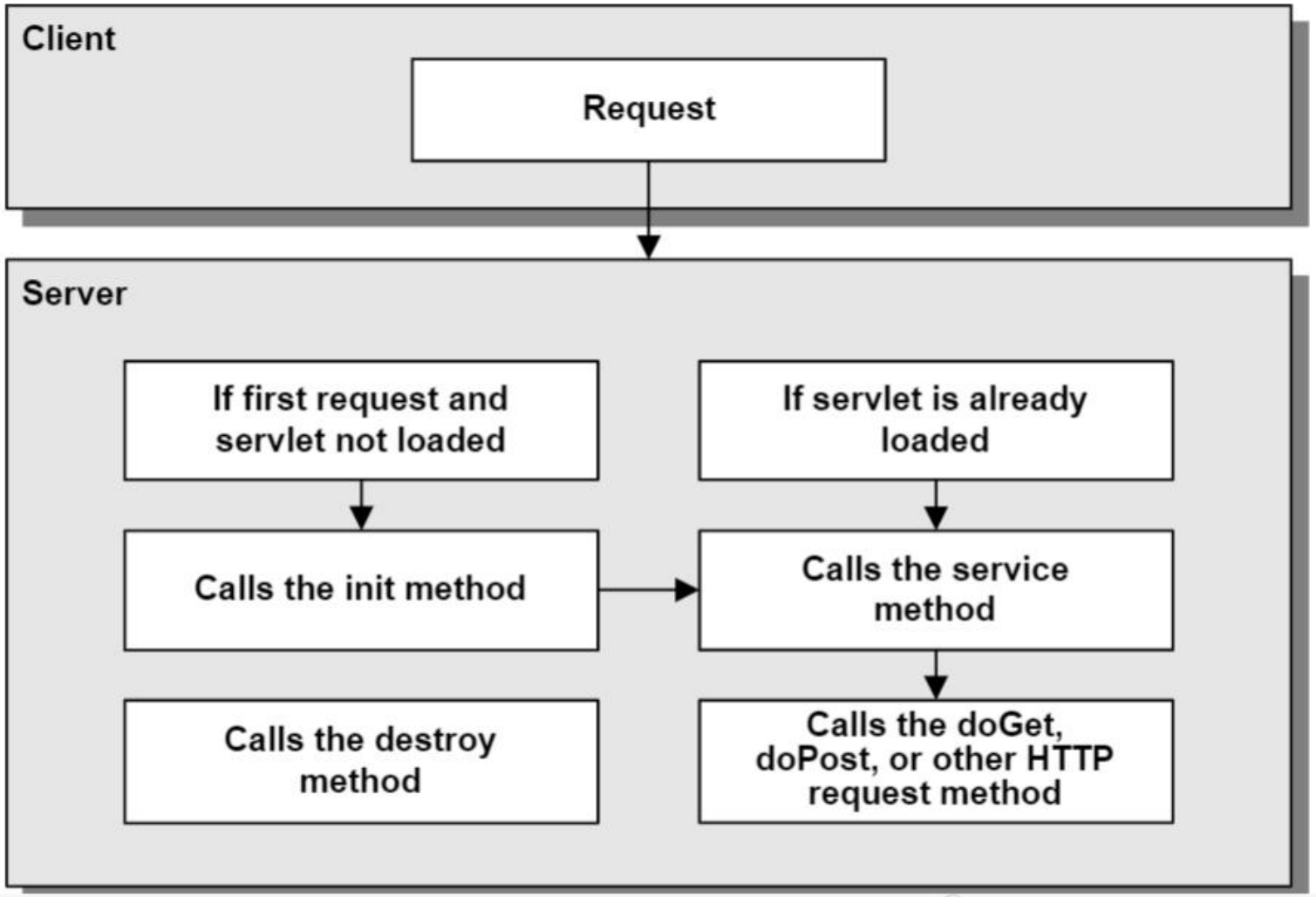
Objectives (continued)

Knowledge

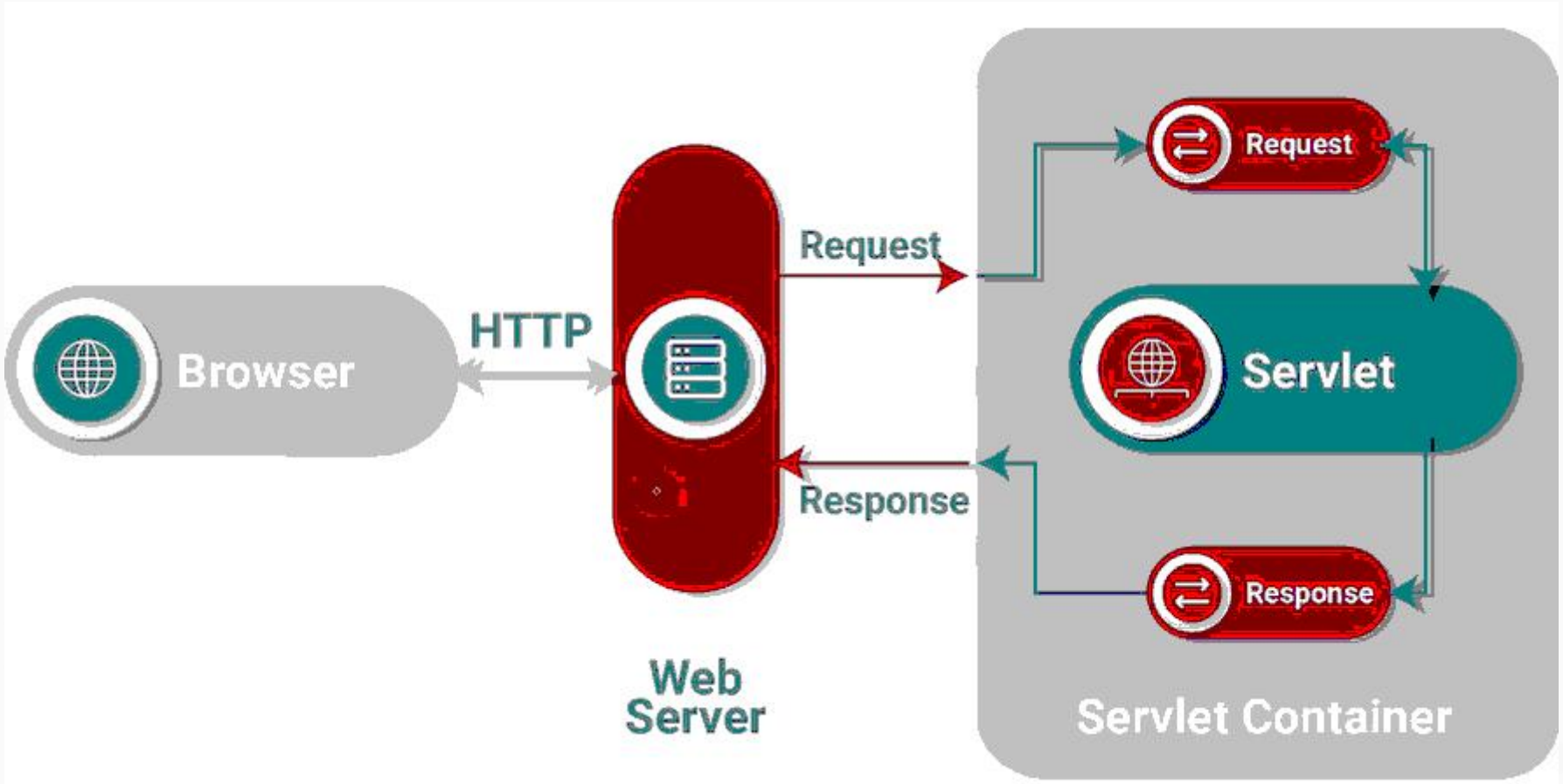
1. Describe servlets and servlet mapping, and their use of request and response objects.
2. Describe how parameters are passed to a servlet with the HTTP GET method.
3. List three reasons for using the HTTP POST method instead of the HTTP GET method.
4. Describe how the ServletContext object is used to get the path for a file. Describe the use of the init, doGet, doPost, and destroy methods in a servlet.
5. Explain why you should never use instance variables in servlets.
6. Describe the use of debugging data written to the console or log file.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

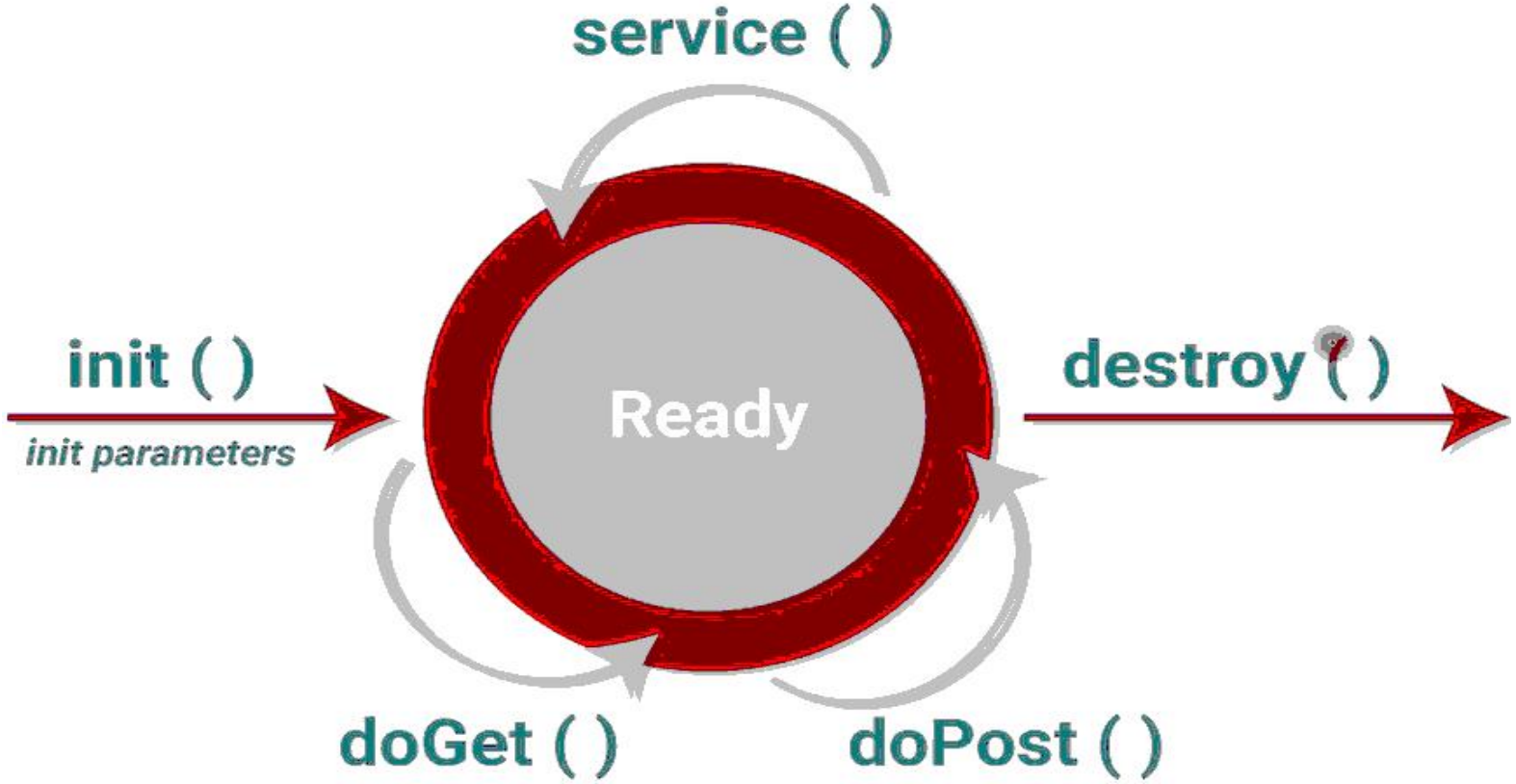
The lifecycle of a servlet



The lifecycle of a servlet



The lifecycle of a servlet



Note

It's generally considered a bad practice to override the service method. Instead, we should override a method like doGet or doPost to handle a specific type of HTTP request.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

The lifecycle of a servlet

init()

invoked only once (when servlet is first instantiated).

perform setup and initialization in this method (ie setting the database connection)

destroy()

invoked just before servlet instance destroyed.

perform the clean-up operations here (ie close the previously created database connection)

service()

do not override this method.

destroy()

invoked just before servlet instance destroyed.

doGet() and doPost() methods

handle the HTTP GET and POST requests

override these 2 methods in your servlet to provide the desired behavior.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Servlet: BMIServlet.java

```
public class BMIServlet extends HttpServlet {
        private static final long serialVersionUID = 1L;

        protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

                String name = request.getParameter("fullname");
                int yob = Integer.parseInt(request.getParameter("yob"));
                double weight = Double.parseDouble(request.getParameter("weight"));
                double height = Double.parseDouble(request.getParameter("height"));
                int age = 2022 - yob;
                double BMI = weight/(height*height);

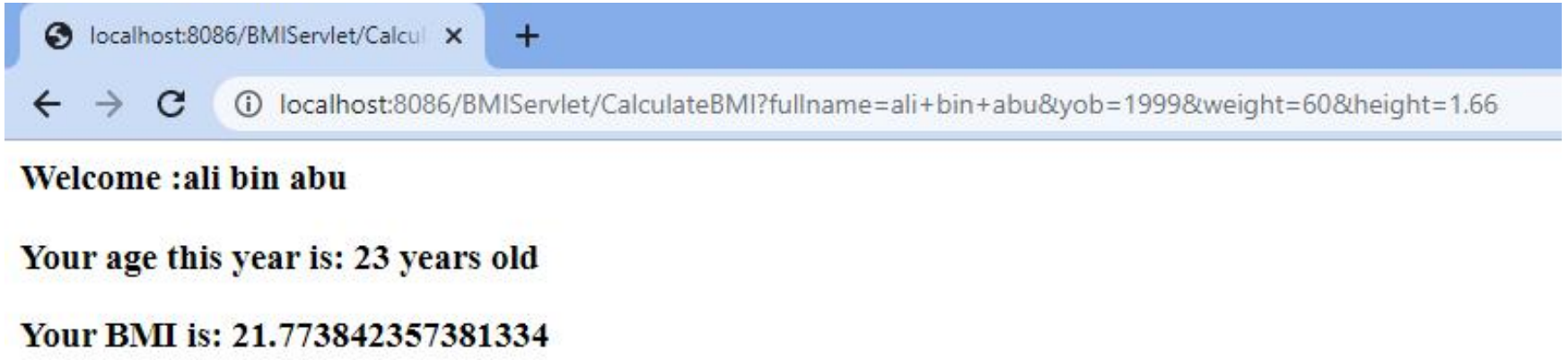
                PrintWriter pw = response.getWriter();
                pw.write("<h3> Welcome :" +name +"</h3>");
                pw.write("<h3>Your age this year is: " +age +" years old </h3>");
                pw.write("<h3>Your BMI is: "+BMI +"</h3>");
        }
}
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

URL request :

```
http://localhost:8086/BMIServlet/CalculateBMI?fullname=ali+samad
&yob=1999&weight=60&height=1.66
```

Result



Servlet concepts

The doGet method processes all HTTP requests that use the GET method.

The doPost method processes all HTTP requests that use the POST method.

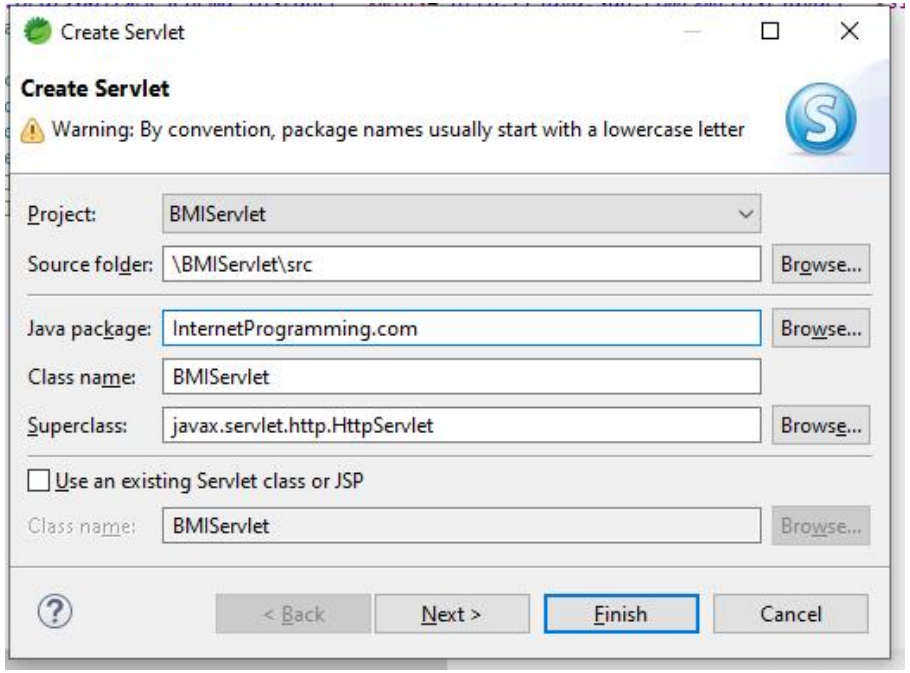
The doGet and doPost methods both accept (1) the HttpServletRequest object, or the *request object*, and (2) the HttpServletResponse object, or the *response object*.

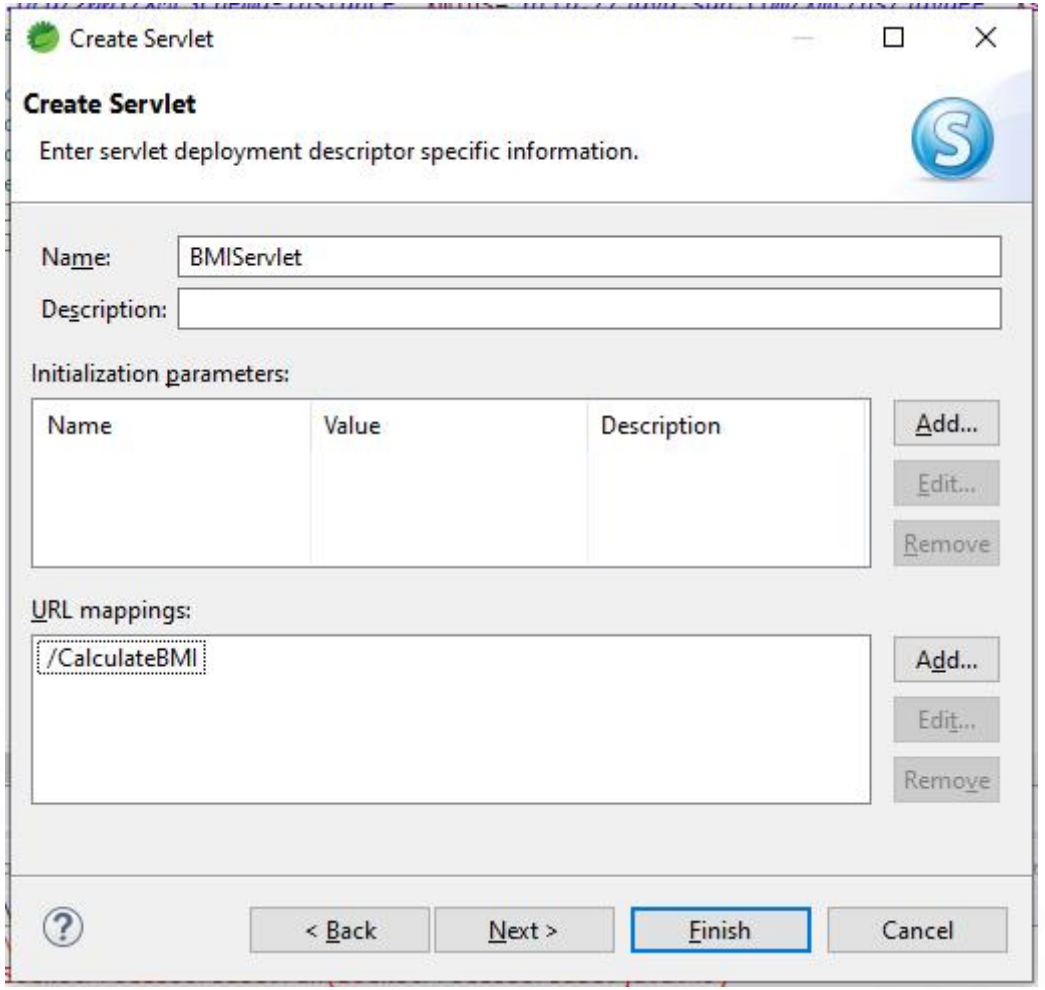
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

XML elements for working with servlet mapping

Element	Description
<servlet-class>	Specifies the class for the servlet.
<servlet-name>	Specifies a unique name for servlet used to identify the servlet within the web.xml file.
<url-pattern>	Specifies URL or URLs mapped to specified servlet.

UTM UNIVERSITI TEKNOLOGI MALAYSIA


Create Servlet
Create Servlet
Warning: By convention, package names usually start with a lowercase letter
Project:
BMIServlet
Source folder:
\BMIServlet\src
Browse...
Java package:
InternetProgramming.com
Browse...
Class name:
BMIServlet
Superclass:
javax.servlet.http.HttpServlet
Browse...
Use an existing Servlet class or JSP
Class name:
BMIServlet
Browse...
< Back
Next >
Finish
Cancel


Create Servlet
Create Servlet
Enter servlet deployment descriptor specific information.
Name:
BMIServlet
Description:
Initialization parameters:
Name
Value
Description
Add...
Edit...
Remove
URL mappings:
/CalculateBMI
Add...
Edit...
Remove
< Back
Next >
Finish
Cancel

XML tags that add servlet mapping to web.xml file

```
<web-app>
  <servlet>
    <description></description>
    <display-name>BMIServlet</display-name>
    <servlet-name>BMIServlet</servlet-name>
    <servlet-class>InternetProgramming.com.BMIServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>BMIServlet</servlet-name>
    <url-pattern>/CalculateBMI</url-pattern>
  </servlet-mapping>
</web-app>
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

or alternatively, use the annotation style @servlet (supported in servlet 3.0 and above)

An annotation that maps a servlet to a URL

```
@WebServlet("/CalculateBMI")
public class BMIServlet extends HttpServlet {
...
...
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

How to use servlet annotations

To map a servlet to multiple URLs

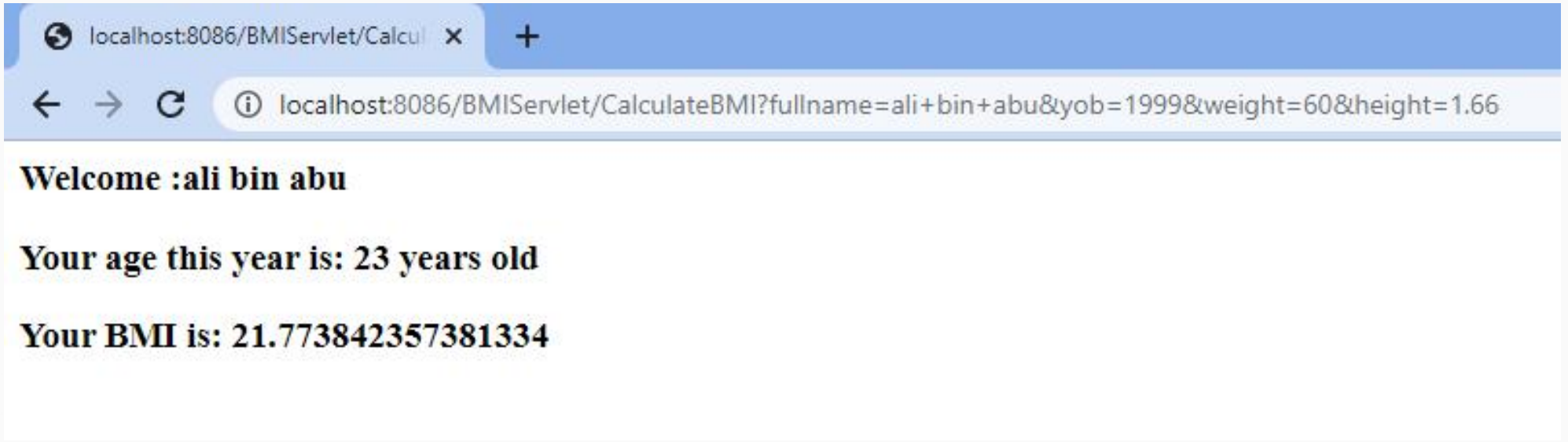
```
@WebServlet(urlPatterns={"/emailList", "/email/*"})
```

To specify an internal name for the servlet

```
@WebServlet(name="BMIServlet", urlPatterns={"/CalculateBMI"})
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

An HTTP request that uses the GET method



How to append parameters to the URL

```
BMICalculate?fullname=ali+abu&yob=1999&weight=60&height=1.66
```

How to append parameters to a GET request

1: Enter the URL into the browser's address bar

```
http://localhost:8086/BMIServlet/CalculateBMI?fullname=ali+samad
&yob=1999&weight=60&height=1.66
```

2: Code a form that uses the GET method

```
<form action="CalculateBMI"> -or-  <form action="CalculateBMI" method="get">
```

3: Code an anchor tag

```
<a href="CalculateBMI?fullname=ali+abu&yob=1999&weight=60&height=1.66">Calculate By BMI</a>
```

Note: the servlet must implement the doGet method to process an HTTP GET request.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

A Form tag that uses the POST method

```
<form action="CalculateBMI" method="post">
```

Use the GET method when...

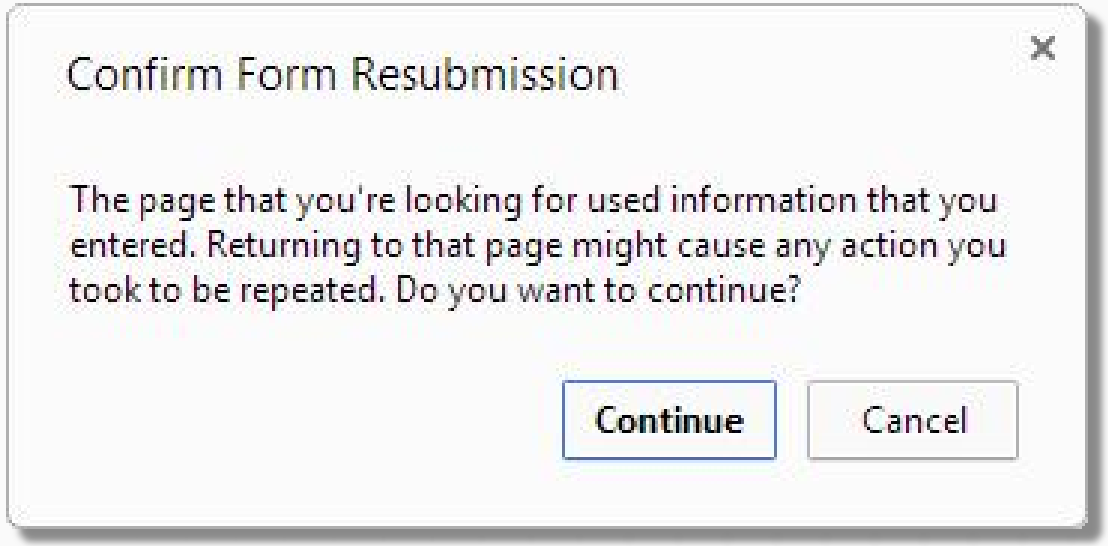
- Request reads data from the server.
- Request can be executed multiple times without causing problems.

Use the POST method when...

- Request writes data to the server.
- Executing request multiple times may cause problems.
- You don't want to include the parameters in the URL for security reasons.
- You don't want users to be able to include parameters when they bookmark a page.
- You need to transfer more than 4 KB of data.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

A typical browser dialog that's displayed if the user tries to refresh a post



Two methods of the HttpServletRequest object

Method	Description
getParameter(String param**)**	Returns the value of the specified parameter as a string if it exists or null if it doesn't. Often, this is the value defined in the value attribute of the control in the HTML or JSP file.
getParameterValues(String param**)**	Returns an array of String objects containing all values that the given request parameter has or null if the parameter doesn't have any values.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Servlet code that gets text from a text box

```
String fullname= request.getParameter("fullname");
```

Servlet code that determines if a box is checked

```
// returns the value or "on" if checked, null otherwise. String rockCheckBox
= request.getParameter("rock");
if (rockCheckBox != null)
{
    // rock music was checked
}
```

Servlet code that reads and processes multiple values from a list box

```
// returns the values of the items selected in a list box.
String[] selectedCountries = request.getParameterValues("country"); for (String
country : selectedCountries)
{
    // code that processes each country
}
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

A method of the GenericServlet class

A method of the ServletContext class

Method	Description
`getRealPath(String path)`	Returns a String object for the absolute path of the specified relative path.

Method	Description
`getServletContext()`	Returns a ServletContext object that contains information about the application's context.

Code that gets the absolute path for a file

```
ServletContext sc = this.getServletContext();
String path = sc.getRealPath("/WEB-INF/EmailList.txt");
```

A more concise way to write the same code

```
String path = this.getServletContext()
                  .getRealPath("/WEB-INF/EmailList.txt");
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

How to work with the ServletContext object

Because servlets inherit the GenericServlet class, the getServletContext method is available to all servlets.

You can use the ServletContext object to read global initialization parameters, to work with global variables, and to write data to log files.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Two methods of the HttpServletRequest object

Method	Description
setAttribute(String name, Object o**)**	Stores any object in the request as an attribute and specifies a name for the attribute. Attributes reset between requests.
getAttribute(String name**)**	Returns value of the specified attribute as an Object type. If no attribute exists for specified name, method returns a null value.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

How to set a request attribute

```
User user = new User(firstName, lastName, email);
request.setAttribute("user", user);
```

How to get a request attribute

```
User user = (User) request.getAttribute("user");
```

How to set a request attribute for a primitive type

```
int id = 1;
request.setAttribute("id", new Integer(id));
```

How to get a request attribute for a primitive type

```
int id = (Integer) request.getAttribute("id");
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

A method of the ServletContext object

A method of the RequestDispatcher object

Method	Description
`forward(ServletRequest request, ServletResponse response)`	Forwards the request and response objects to another resource on the server (usually a JSP or servlet).

Method	Description
`getRequestDispatcher(String path)`	Returns a RequestDispatcher object for specified path.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

How to forward the request to an HTML page

```
String url = "/index.html";
getServletContext().getRequestDispatcher(url)
    .forward(request, response);
```

How to forward the request to a JSP

```
String url = "/thanks.jsp";
getServletContext().getRequestDispatcher(url)
    .forward(request, response);
```

How to forward the request to a servlet

```
String url = "/cart/displayInvoice";
getServletContext().getRequestDispatcher(url)
     .forward(request, response);
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

A method of the HttpServletResponse class

Method	Description
`sendRedirect(``String path`**`)`**	Sends a temporary redirect response to the client using the specified redirect location URL.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

…relative to the current directory

```
response.sendRedirect("email");
```

…relative to the servlet engine

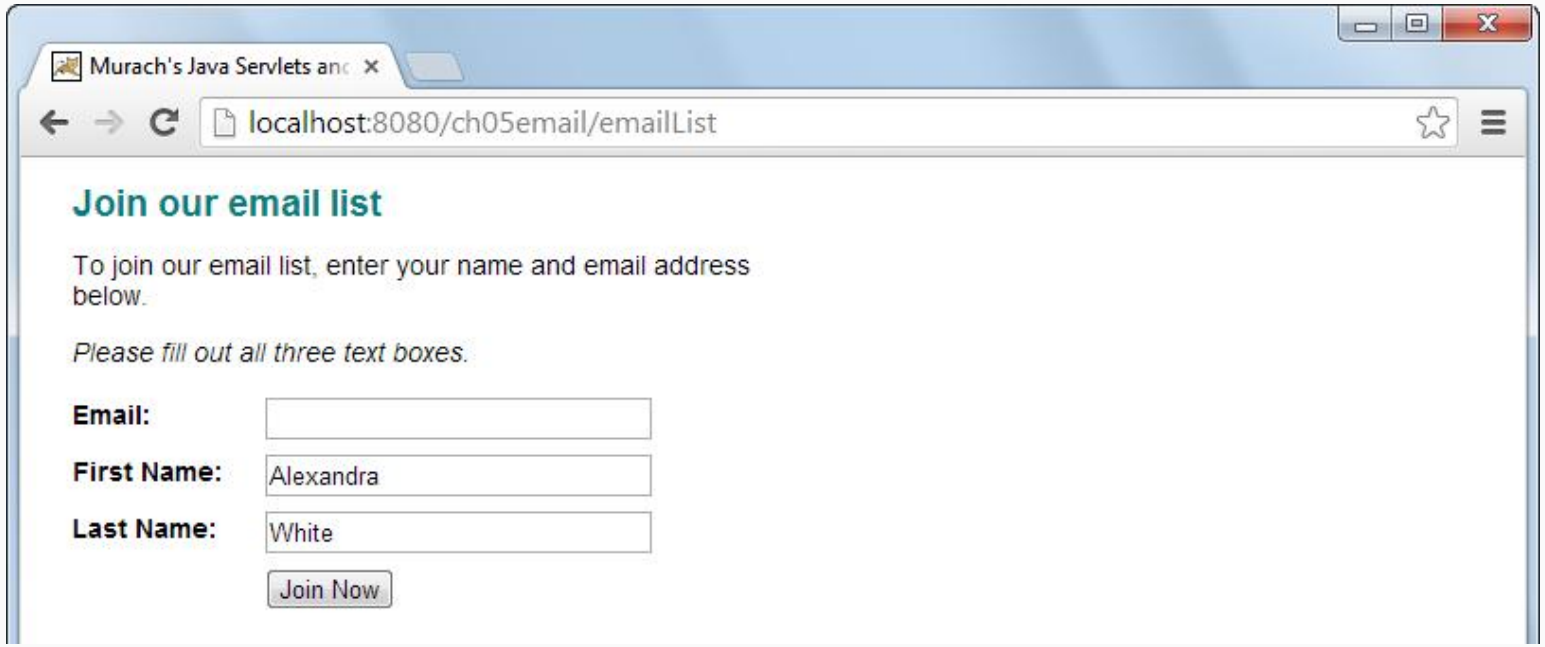
```
response.sendRedirect("/BMIApp/email/");
```

…to a different web server

```
response.sendRedirect("http://google.com");
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

The JSP displayed when an entry isn't made



The code for the JSP

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Murach's Java Servlets and JSP</title>
    <link rel="stylesheet" href="styles/main.css" type="text/css"/>
</head>
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

The code for the JSP (cont)

```
<body>
    <h1>Join our email list</h1>
    <p>To join our email list, enter your name and
       email address below.</p>
    <p><i>${message}</i></p>
    <form action="emailList" method="post">
        <input type="hidden" name="action" value="add">
        <label class="pad_top">Email:</label>
        <input type="email" name="email" value="${user.email}"><br>
        <label class="pad_top">First Name:</label>
        <input type="text" name="firstName"
               value="${user.firstName}"><br>
        <label class="pad_top">Last Name:</label>
        <input type="text" name="lastName"
               value="${user.lastName}"><br>
        <label> </label>
        <input type="submit" value="Join Now" class="margin_left">
    </form>
</body>
</html>
```

```
@Override
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
                      throws ServletException, IOException {

    String url = "/index.jsp";

    // get current action
    String action = request.getParameter("action");
    if (action == null) {
        action = "join";  // default action
    }

    // perform action and set URL to appropriate page
    if (action.equals("join")) {
        url = "/index.jsp";    // the "join" page
    }
    else if (action.equals("add")) {
        // get parameters from the request
        String firstName = request.getParameter("firstName");
        String lastName = request.getParameter("lastName");
        String email = request.getParameter("email");
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

A doPost method that validates data (continued)

```
        // store data in User object
        User user = new User(firstName, lastName, email);

        // validate the parameters
        String message;
        if (firstName == null || lastName == null ||
            email == null ||
            firstName.isEmpty() || lastName.isEmpty() ||
            email.isEmpty()) {

            message = "Please fill out all three text boxes.";
            url = "/index.jsp";
        } else {
            message = "";
            url = "/thanks.jsp";
            UserDB.insert(user);
        }
        request.setAttribute("user", user);
        request.setAttribute("message", message);
    }
    getServletContext()
            .getRequestDispatcher(url)
            .forward(request, response);
}
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

A complete web.xml file

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
             http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">

    <context-param>
        <param-name>custServEmail</param-name>
        <param-value>custserv@murach.com</param-value>
    </context-param>

    <servlet>
        <servlet-name>EmailListServlet</servlet-name>
        <servlet-class>murach.email.EmailListServlet</servlet-class>
        <init-param>
            <param-name>relativePathToFile</param-name>
            <param-value>/WEB-INF/EmailList.txt</param-value>
        </init-param>
    </servlet>

    <servlet-mapping>
        <servlet-name>EmailListServlet</servlet-name>
        <url-pattern>/emailList</url-pattern>
    </servlet-mapping>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

```
    <!-- you can comment out these error tags
         when the app is in development -->
    <error-page>
        <error-code>404</error-code>
        <location>/error_404.jsp</location>
    </error-page>
    <error-page>
        <exception-type>java.lang.Throwable</exception-type>
        <location>/error_java.jsp</location>
    </error-page>

    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>

    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>
</web-app>
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Concepts about the web.xml file

The web.xml file is stored in the WEB-INF directory. When Tomcat starts, it reads the web.xml file.

If elements in the web.xml aren't nested correctly, Tomcat displays an error message when it reads the web.xml file.

After you modify the web.xml file, redeploy the application so the changes take effect. Or, restart Tomcat.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Sub-Topic: Servlet Initialization parameters

XML elements for initialization parameters

Element	Description
`<context-param>`	Defines a parameter that's available to all servlets within an application.
`<servlet>`	Identifies a specific servlet within the application.
`<servlet-name>`	Defines the name for the servlet that's used in the rest of the web.xml file.
`<servlet-class>`	Identifies the servlet by specifying the servlet's package and class name.
`<init-param>`	Defines a name/value pair for an initialization parameter for a servlet.
`<param-name>`	Defines the name of a parameter.
`<param-value>`	Defines the value of a parameter.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Initialization parameters in a web.xml file

```
<context-param>
    <param-name>custServEmail</param-name>
    <param-value>custserv@murach.com</param-value>
</context-param>

<servlet>
    <servlet-name>AddToEmailListServlet</servlet-name>
    <servlet-class>email.AddToEmailListServlet</servlet-class>
    <init-param>
        <param-name>relativePathToFile</param-name>
        <param-value>/WEB-INF/EmailList.txt</param-value>
    </init-param>
</servlet>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

An annotation that sets initialization parameters for a servlet

```
@WebServlet(urlPatterns={"/emailList"},
            initParams={@InitParam(name="relativePathToFile",
                                   value="/WEB-INF/EmailList.txt")})
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Initialization parameters

To create an *initialization parameter* available to all servlets (called a *context initialization parameter*), code the param-name and param-value elements within the context-param element.

To create an initialization parameter available to a specific servlet (called a *servlet initialization parameter*), code the param-name and param-value elements within the init-param element.

But first, identify the servlet by coding the servlet, servlet-name, and servlet-class elements.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Two methods of the GenericServlet class

Method	Description
`getServletContext()`	Returns a ServletContext object that contains information about the entire web application's context.
`getServletConfig()`	Returns a ServletConfig object that contains information about a single servlet's configuration.

A method of the ServletContext and ServletConfig interfaces

Method	Description
`getInitParameter(String name)`	Returns a String object that contains the value of the specified initialization parameter. If the parameter doesn't exist, this method returns a null value.

Code that gets an initialization parameter that's...

...available to all servlets

```
String custServEmail = this.getServletContext()
                           .getInitParameter("custServEmail");
```

...available to the current servlet only

```
String relativePath = this.getServletConfig()
                          .getInitParameter("relativePathToFile");
```

Sub-Topic: Handling Errors

XML elements for working with error handling

Element	Description
`<error-page>`	Specifies an HTML page or JSP displayed when the application encounters an uncaught exception or a certain type of HTTP status code.
`<error-code>`	Specifies the number of a valid HTTP status code.
`<exception-type>`	Uses the fully qualified class name to specify a Java exception.
`<location>`	Specifies the location of the HTML page or JSP that's displayed.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

How to provide error-handling for an HTTP 404 status code

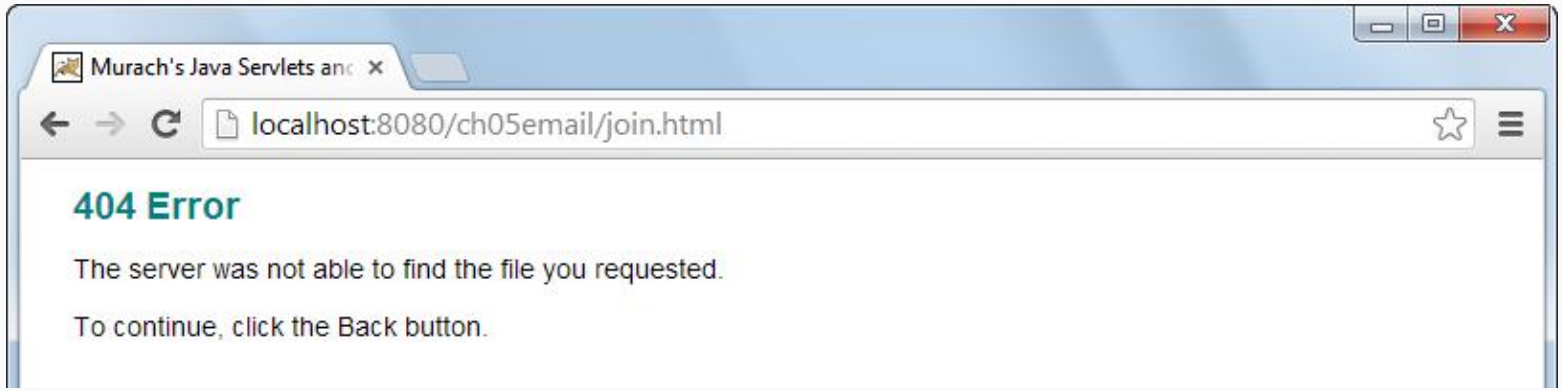
The XML tags

```
<error-page>
    <error-code>404</error-code>
    <location>/error_404.jsp</location>
</error-page>
```

The error_404.jsp file

```
<h1>404 Error</h1>
<p>The server was not able to find the file you requested.</p>
<p>To continue, click the Back button.</p>
```

The JSP page for the 404 error



How to provide error-handling for all Java exceptions

The XML tags

```
<error-page>
    <exception-type>java.lang.Throwable</exception-type>
    <location>/error_java.jsp</location>
</error-page>
```

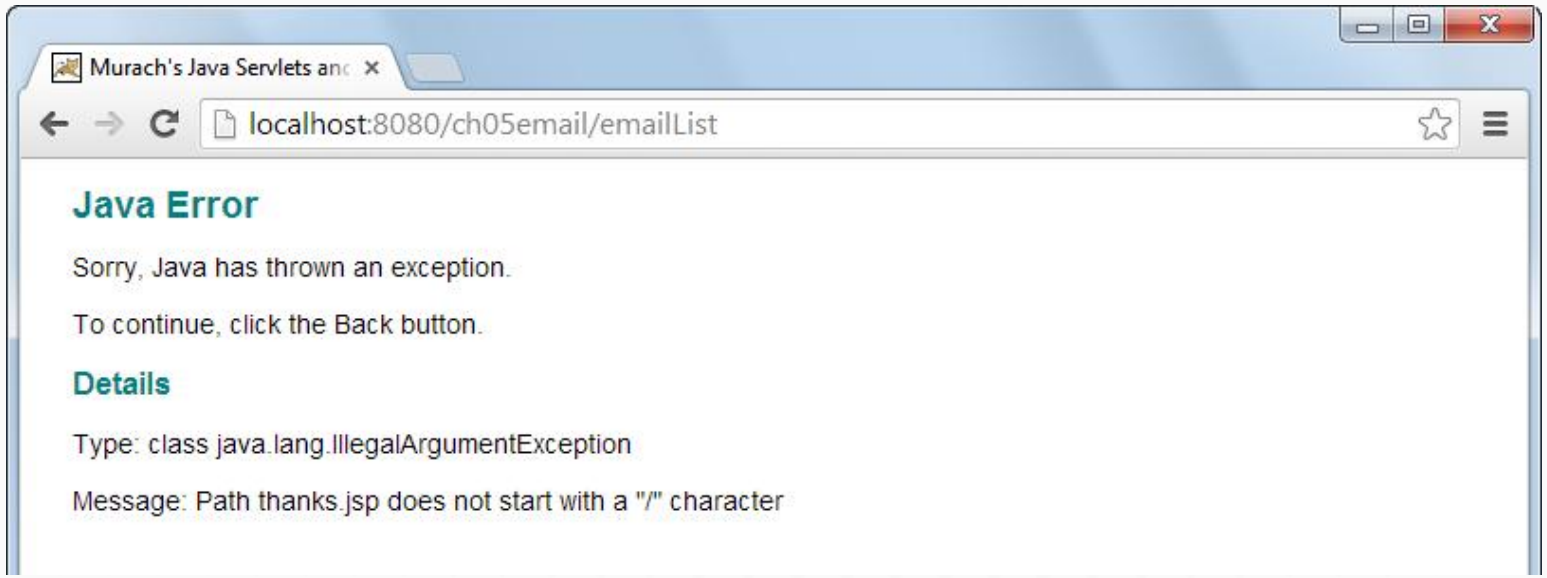
The error_java.jsp file

```
<h1>Java Error</h1>
<p>Sorry, Java has thrown an exception.</p>
<p>To continue, click the Back button.</p>

<h2>Details</h2>
<p>Type: ${pageContext.exception["class"]}</p>
<p>Message: ${pageContext.exception.message}</p>
```

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

The JSP when a Java exception is thrown



Five common methods of a servlet

```
public void init() throws ServletException

public void service(HttpServletRequest request,
                    HttpServletResponse response)
                    throws IOException, ServletException

public void doGet(HttpServletRequest request,
                  HttpServletResponse response)
                  throws IOException, ServletException

public void doPost(HttpServletRequest request,
                   HttpServletResponse response)
                   throws IOException, ServletException

public void destroy()
```

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

A servlet with an instance variable

```
package murach.email;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class EmailListServlet extends HttpServlet {

    // declare an instance variable for the page
    private int globalCount; // not thread-safe

    @Override
    public void init() throws ServletException
    {
        globalCount = 0; // initialize the instance variable
    }
```

```
    @Override
    protected void
        doPost(  HttpServletRequest
        request,  HttpServletResponse
        response)
        throws ServletException, IOException
    {
        globalCount++;    // this is not thread-safe

        // the rest of the code goes here
    }
}
```

Instance Variable Concerns

An *instance variable* of a servlet belongs to the one instance of the servlet and is shared by any threads that request the servlet.

Instance variables are not *thread-safe*. Two threads may conflict when they try to read, modify, and update the same instance variable at the same time, which can result in lost updates or other problems.

Common servlet problems

Problem	Possible solutions
The servlet won't compile	Make sure compiler has access to the JAR files for all necessary APIs.
The servlet won't run	Make sure web server is running. Make sure URL is correct.
Changes aren't showing up	Redeploy the application. Restart the server so it reloads all applications. Make sure servlet reloading is on.
The page doesn't display correctly	Use browser to view the HTML code for the page. Go through the HTML code to identify the problem, and fix the problem in the servlet.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Sub-Topic: Print Debugging Data

How to print debugging data to the console

Use the println method of the System.out or System.err objects to display debugging messages on the console for the servlet engine.

When you use debugging messages to display variable values, include servlet name and variable name so messages are easy to interpret.

NetBeans displays a tab for the Tomcat console within its Output window.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Code that prints debugging data to the console

```
public void doPost(HttpServletRequest request,
                   HttpServletResponse  response)
                   throws IOException, ServletException
{
    // code

    String email = request.getParameter("email");
    System.out.println("EmailListServlet email: " + email);

    // more code
}
```

Sample output displayed on the Tomcat console

```
EmailListServlet email: jsmith@gmail.com
```

Two methods of the HttpServlet class

Method	Description
`log(String message)`	Writes the specified message to the server's log file.
`log(String message, Throwable t)`	Writes the specified message to the server's log file, followed by the stack trace for the exception.

Servlet code that prints a variable to a log file

```
log("email=" + email);
```

Sample data that's written to the log file

```
Jun 29, 2014 6:26:04 PM org.apache.catalina.core.ApplicationContext
log
INFO: EmailListServlet: email=jsmith@gmail.com
```

Servlet code that prints a stack trace to a log file

```
try {
    UserIO.add(user, path);
} catch(IOException e) {
    log("An IOException occurred.", e);
}
```

The data that's written to the log file

```
Jun 29, 2014 6:30:38 PM
org.apache.catalina.core.StandardWrapperValve invoke
WARNING: Servlet.service() for servlet EmailListServlet threw
exception
java.io.FileNotFoundException:
C:\murach\servlet_and_jsp\netbeans\ch05email\build\web\WEB-
INF\EmailList.txt (Access is denied)
        at java.io.FileOutputStream.openAppend(Native Method)
        at
java.io.FileOutputStream.<init>(FileOutputStream.java:177)
        at java.io.FileWriter.<init>(FileWriter.java:90)
        at murach.data.UserIO.add(UserIO.java:11)
        at
murach.email.EmailListServlet.doPost(EmailListServlet.java:38)
...
...
```

The location of a typical Tomcat log file

```
C:\tomcat-8.0\logs\localhost.2014-06-29.log
```

How to print debugging data to a log file

Use the log methods of the HttpServlet class to write debugging data to web server's *log file*.

Stack trace is the chain of method calls for any statement that calls a method.

Data that's written by log methods varies from one server to another. The name and location of the log files may also vary from one server to another.

NetBeans displays a tab for the server log within its Output window.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

TOPIC 2 – Java Servlet

the end



UTM JOHOR BAHRU

innovative • entrepreneurial • global